HelpDesk Service Desk Application

A complete IT service desk application built with Docker, similar to JIRA for managing IT support tickets.

Features

- **Ticket Management**: Create, view, update, and track IT support tickets
- **User Management**: Multiple users with role-based access
- **File Attachments**: Upload screenshots and documents with tickets
- **Priority Levels**: Low, Medium, High, Critical
- **Status Tracking**: Open, In Progress, Resolved, Closed
- **Categories**: Hardware, Software, Network, Access, Other
- **Dashboard**: Real-time statistics and metrics
- **Responsive UI**: Modern React-based interface

Technology Stack

- **Frontend**: React, Tailwind CSS
- **Backend**: Node.js, Express
- **Database**: PostgreSQL
- **Containerization**: Docker & Docker Compose
- **Web Server**: Nginx

Project Structure

```
helpdesk-app/
├── docker-compose.yml
├── init.sql
├── backend/
│   ├── Dockerfile
│   ├── package.json
│   ├── server.js
│   └── uploads/
└── frontend/
    ├── Dockerfile
    ├── package.json
    ├── nginx.conf
    ├── public/
    │   └── index.html
    └── src/
        ├── App.js
        └── index.js
```

Prerequisites

- Docker installed (version 20.10 or higher)
- Docker Compose installed (version 2.0 or higher)
- 4GB RAM minimum
- Ports 80, 3000, and 5432 available

Installation & Setup

Step 1: Create Project Directory

bash

```bash
mkdir helpdesk-app
cd helpdesk-app
```

Step 2: Create Directory Structure

bash

```
mkdir -p backend/uploads
mkdir -p frontend/src
mkdir -p frontend/public
```

Step 3: Create Configuration Files

1. **Copy `docker-compose.yml`** to root directory
2. **Copy `init.sql`** to root directory
3. **Copy backend files**:
   - `backend/Dockerfile`
   - `backend/package.json`
   - `backend/server.js`
4. **Copy frontend files**:
   - `frontend/Dockerfile`
   - `frontend/package.json`
   - `frontend/nginx.conf`

Step 4: Create Frontend Files

Create `frontend/public/index.html`:

html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>HelpDesk Service Desk</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body>
  <div id="root"></div>
</body>
</html>
```

Create `frontend/src/index.js`:

javascript

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Create `frontend/src/App.js` with the React component code provided earlier.

Step 5: Build and Start the Application

bash

```bash
# Build all containers
docker-compose build

# Start all services
docker-compose up -d

# View logs
docker-compose logs -f
```

Step 6: Access the Application

- **Frontend**: http://localhost
- **Backend API**: http://localhost:3000
- **Database**: localhost:5432

Usage Guide

Creating a New Ticket

1. Click the "New Ticket" button
2. Fill in the ticket details:
    - Title: Brief summary of the issue
    - Description: Detailed explanation
    - Priority: Select appropriate level
    - Category: Choose issue type
3. Optionally attach files (screenshots, logs, etc.)
4. Click "Create Ticket"

Managing Tickets

1. View all tickets in the "Tickets" tab
2. Filter by status, priority, or category
3. Click on a ticket to view details
4. Update status and assign to team members
5. Add comments for collaboration

Dashboard

- View real-time statistics
- Monitor open and resolved tickets
- Track high-priority issues
- Analyze ticket distribution

Docker Commands

Start Application

bash

```
docker-compose up -d
```

Stop Application

bash

```
docker-compose down
```

View Logs

bash

```bash
# All services
docker-compose logs -f

# Specific service
docker-compose logs -f backend
docker-compose logs -f frontend
docker-compose logs -f db
```

Rebuild After Changes

bash

```bash
docker-compose down
docker-compose build
docker-compose up -d
```

Access Database

bash

```bash
docker exec -it helpdesk_db psql -U helpdesk_user -d helpdesk
```

Remove All Data (Reset)

bash

```bash
docker-compose down -v
docker-compose up -d
```

API Endpoints

Users

- `GET /api/users` - List all users
- `POST /api/users` - Create new user

Tickets

- `GET /api/tickets` - List tickets (with filters)
- `GET /api/tickets/:id` - Get ticket details
- `POST /api/tickets` - Create new ticket
- `PUT /api/tickets/:id` - Update ticket
- `POST /api/tickets/:id/comments` - Add comment

Dashboard

- `GET /api/dashboard/stats` - Get statistics

Database Schema

Tables

- **users**: User accounts and profiles
- **tickets**: IT support tickets
- **attachments**: File uploads
- **comments**: Ticket discussions

Troubleshooting

Port Already in Use

bash

```bash
# Check what's using the port
lsof -i :80
lsof -i :3000

# Change ports in docker-compose.yml
```

Database Connection Issues

bash

```
# Check database health
docker-compose ps

# Restart database
docker-compose restart db
```

Frontend Not Loading

bash

```
# Check frontend logs
docker-compose logs frontend

# Rebuild frontend
docker-compose build frontend
docker-compose up -d frontend
```

File Upload Issues

bash

```
# Check permissions
docker exec helpdesk_backend ls -la /app/uploads

# Recreate uploads volume
docker-compose down -v
docker-compose up -d
```

Development Mode

For development with hot-reload:

bash

```
# Backend
cd backend
npm install
npm start

# Frontend
cd frontend
npm install
npm start
```

Production Deployment

1. Update environment variables
2. Use production database credentials
3. Enable HTTPS with SSL certificates
4. Configure proper CORS settings
5. Set up backup strategy for PostgreSQL

Future Enhancements

- Email notifications
- Authentication & authorization
- SLA tracking
- Knowledge base
- Reporting & analytics
- Mobile app
- Integration with other tools

License

MIT License

Support

For issues and questions, create a new ticket in the system!